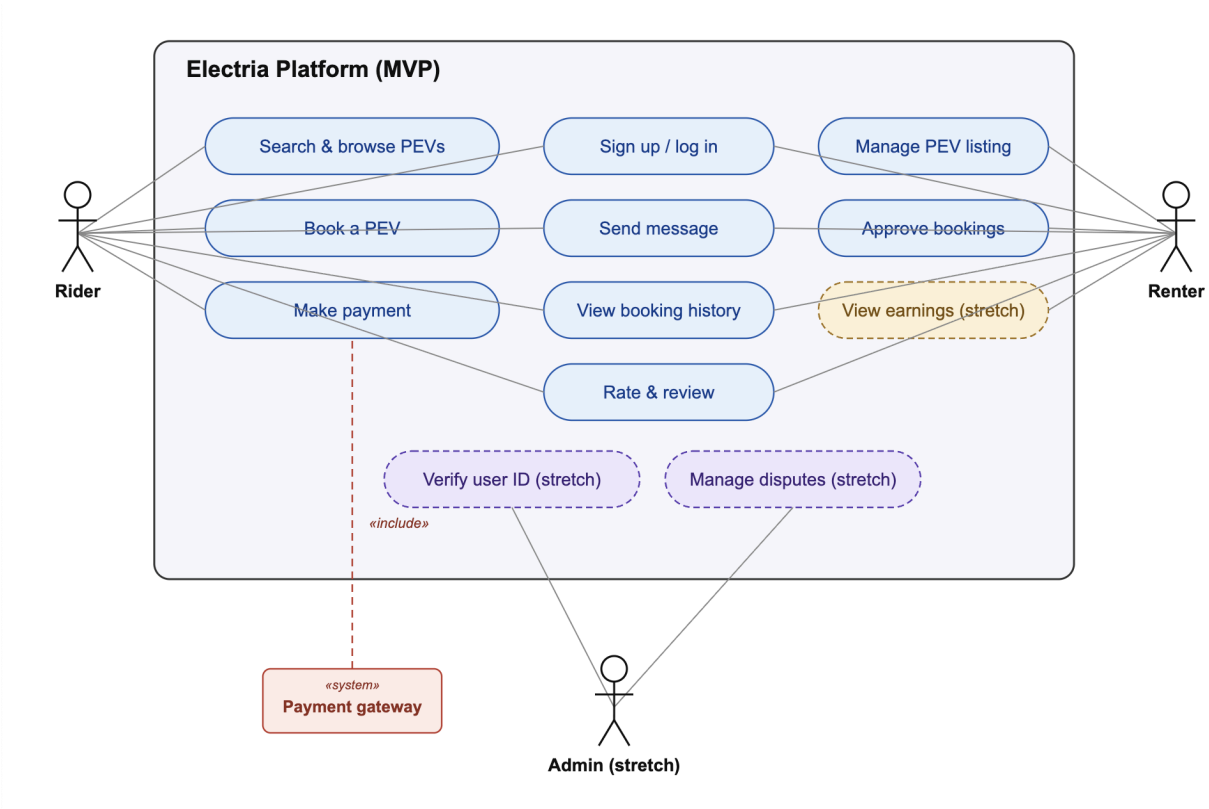
Project Name: Electria (Group 15)

Use Case Diagram



Functional Requirements:

1. Users shall register and log in as a Rider, Renter, or both.
2. Renters shall create a PEV listing with photos, category, price, and availability.
3. Riders shall search and filter PEVs by location, price, and vehicle type.
4. Riders shall submit a booking request for a date/time range; Renters shall approve or decline it.
5. The system shall process a test-mode payment when a booking is confirmed.
6. Riders and Renters shall exchange messages tied to a specific booking.
7. The system shall track booking status (requested, confirmed, completed, cancelled).
8. Riders and Renters shall rate and review each other after a completed rental.
9. Users shall view their own booking history.

Stretch Functional Requirements:

1. Admins shall be able to mark a user's ID as verified.
2. Renters shall maintain a maintenance/incident log per PEV.
3. Renters shall view an earnings summary across completed bookings.
4. Admins shall view flagged accounts and resolve disputes.
5. The system shall support live (non-polling) updates for messages and booking status.

Non-Functional Requirements

Performance	Search results shall return within 2 seconds for a typical query against the demo dataset.
Security	Passwords shall be hashed, not stored in plaintext; all traffic shall use HTTPS.
Usability	A new user shall be able to sign up and complete a search without external help.
Compatibility	The app shall work correctly in the current versions of Chrome, Safari, and Firefox.
Reliability	A failed payment shall not leave a booking marked as confirmed.
Maintainability	The codebase shall follow a consistent schema (via Prisma) and folder structure so any teammate can add a feature without breaking another's.
Data integrity	Booking and payment records shall be stored transactionally so partial writes can't corrupt a booking's state.
Scalability	Not a priority for this project — the schema should be reasonable, but no infrastructure work is needed to "scale" a class demo.

Technologies

Frontend: Next.js (React + TypeScript)
Backend: Next.js API routes (same codebase, no separate server)

Database: PostgreSQL (Supabase or Neon), accessed via Prisma
Auth: NextAuth.js (or Supabase Auth)
Payments: Stripe
Hosting: Vercel